JIMMA CITY ADMINISTRATION

Jimma City Smart Asset Management System

Technical Design and Specification

Technology, code structure, API, database, configuration and build

This document gives the technical detail behind the design: the technology used and why, how the code is organised, every API endpoint, the database tables, the validation and error rules, the configuration settings, and how the system is built, tested and installed. It is meant for the assessment team and for the engineers who will maintain the system.

Document number	JCSAMS-TDD-003
Version	1.0 (submission issue)
Date of issue	25 October 2026
System owner	Jimma City Administration, Ethiopia
Developed by	Banmen Technologies
Prepared by	Bealem Sintayehu, Project Lead, Banmen Technologies
Submitted to	Information Network Security Administration (INSA), through the Hibir platform
Assessment requested	Web application and API security assessment (penetration test)
Classification	RESTRICTED

Document control

Revision history

Version	Date	Author	Description of change
0.3	10 October 2026	Bealem Sintayehu	Stack, structure and database sections.
0.7	20 October 2026	Bealem Sintayehu	API reference completed; validation and error model.
1.0	25 October 2026	Bealem Sintayehu	Issued for the INSA assessment; matches migration 020, the AI assistant and the 265-test suite.

Review and approval

This document is valid for submission once the people below have reviewed and signed it. Signatures are collected on the printed copy kept in the project file at the Jimma City Administration.

Role	Name	Position	Signature	Date
Prepared by	Bealem Sintayehu	Project Lead, Banmen Technologies		
Reviewed by		Head, ICT Directorate, Jimma City Administration		
Approved by		Office of the Mayor, Jimma City Administration		

Distribution

Copy	Recipient	Form
1	Information Network Security Administration (INSA), assessment team	Electronic, through the Hibir platform
2	Office of the Mayor, Jimma City Administration	Printed and electronic
3	ICT Directorate, Jimma City Administration	Printed and electronic
4	Project file	Printed, signed original

Related documents in this submission

Number	Title
JCSAMS-LTR-000	Covering letter to INSA
JCSAMS-SRS-001	Software Requirements Specification
JCSAMS-SDD-002	System and Architecture Design Document
JCSAMS-TDD-003	Technical Design and Specification
JCSAMS-UG-004	User Guide
JCSAMS-SEC-005	Security Documentation

Contents

1 Introduction 4

1.1 Purpose and audience 4

1.2 Conventions 4

2 Technology stack 4

3 Source code organisation 5

3.1 Backend modules 5

4 Request handling 6

4.1 Middleware order 6

4.2 Security headers as sent 6

4.3 Authentication 7

5 API reference 7

5.1 Response format 11

6 Database 12

6.1 General rules 12

6.2 Tables 12

6.3 Migrations 13

7 Validation and error handling 14

7.1 Input rules 14

7.2 HTTP status codes 14

8 Configuration 15

9 Background jobs 16

10 Front-end design 16

11 Build, installation and operation 16

11.1 Installation 16

11.2 Operations commands 17

11.3 Testing strategy 17

11.4 Coding conventions 17

1 Introduction

1.1 Purpose and audience

The System and Architecture Design Document (JCSAMS-SDD-002) explains how the system is put together. This document goes one level down. It lists the actual endpoints, tables, settings and commands, so that a tester can find every entry point and a maintainer can find every moving part. Where a value is configurable, the default is given.

1.2 Conventions

- Paths in `monospace` are relative to the project root unless they start with `/api`, in which case they are URL paths.
- `:id` in a path is a numeric record identifier; `:token` is an asset's public token.
- "Signed in" means a valid session token in the `Authorization: Bearer` header. "In area" means the record must be inside the caller's assigned area, otherwise the answer is 404.

2 Technology stack

Layer	Technology	Version	Why it was chosen
Runtime	Node.js	20 LTS or later (tested on 20 and 26)	One language (JavaScript) for client and server; long-term support releases; small footprint on one server.
Web framework	Express	5.2	Mature and widely understood; Express 5 forwards errors from async handlers to the error handler, which removes a common class of unhandled-error bugs.
Database	MySQL / MariaDB	8.0+ / 10.6+	Already known to the City's ICT staff; transactional (InnoDB); supports CHECK constraints and utf8mb4 for Amharic and Afaan Oromo text.
Query builder	Knex	3.3	Bound parameters everywhere; schema migrations in version-controlled files.
Validation	zod	4.6	Declarative schemas with per-field error messages; unknown fields are dropped.
Security headers	helmet	8.3	Standard, maintained set of HTTP security headers including CSP.
Tokens	jsonwebtoken	9.0	HS256 session tokens; no server-side session store needed.
Passwords	bcryptjs	3.0	bcrypt in pure JavaScript; no native build on the server.
Uploads	multer	2.4	Size and count limits enforced while streaming to disk.
Client	React + React Router	19.3 / 7.18	Component model suited to the many forms; client-side routing for a fast single-page application.
Build	Vite	current	Fast production build with hashed file names; no inline scripts (compatible with the strict CSP).
Maps, QR, barcodes	Leaflet; html5-qrcode; qrcode; JsBarcode	1.9; 2.3; 1.5; 3.12	Open-source, no paid service keys.
Testing	node:test (built into Node.js)	-	No extra test framework to maintain; runs the real API against a real test database.

3 Source code organisation

```
jimma-asset-management-system/
  package.json               root scripts: setup, migrate, seed, build, start, test
  backend/
    src/
      server.js              starts the HTTP server and the scheduler
      app.js                 middleware order, route mounting, static files
      config.js              reads and checks environment settings at start-up
      db.js                  Knex connection pool
      middleware/            auth.js (session check), errors.js (error model)
      routes/                18 route modules (one per functional area)
      lib/                   30 domain modules (rules shared by routes)
      migrations/            001 ... 020, applied in order
      scripts/               migrate, seed, create-admin, reset-password,
                             provision-admins, backup, restore
      i18n/                  am.json, om.json (translations), index.js
    test/                    16 test files, helpers.js, env.js
    .env.example             every setting with its default
  frontend/
    public/                  sw.js (service worker), manifest, icons, images
    src/
      main.jsx, App.jsx      start-up, routing, menus per role, layout
      api.js                 the only place that calls the server
      auth.jsx               session state, idle sign-out
      offline.js             outbox for field results
      i18n.jsx               language switching and translation lookup
      components/            shared UI: forms, tables, dialogs, charts, scanner
      pages/                 30 pages (dashboards, assets, approvals, ...)
  docs/                      SECURITY.md and this submission
```

3.1 Backend modules

Module (backend/src/lib)	Responsibility
permissions.js	Loads role permissions from the database (cached, invalidated on change), resolves each user's area, decides which account roles a user manages.
assetGuards.js	`guardAsset` middleware: loads an asset only if it is in the caller's area and the caller has the permission; refuses changes to disposed assets.
assetQuery.js	Builds the scoped, filtered, paged asset query used by the list, export, map and dashboards.
schemas.js, assetSchemas.js	All input rules: names, phones, e-mail, passwords, money, dates, coordinates, enumerations, asset fields.
requests.js, workflow.js	The approval engine for transfers, disposals, retirements, assignments and ownership changes; one- or two-step workflows; separation of duties.
money.js, depreciation.js	Recording valuations and income with their checks; straight-line depreciation.
monthlyReports.js, calendar.js	Who reports to whom, preparing and freezing figures, deadlines, Ethiopian calendar conversion.
alerts.js, notify.js, channels.js	Finding due and overdue items, creating notifications without duplicates, sending e-mail and SMS.
audit.js	Writing audit entries with old and new values.
security.js	Rate limiter, no-store header, bcrypt cost, lockout and idle values, temporary password generation and expiry, file signature checks.

Module (backend/src/lib)	Responsibility
publicAsset.js	Public token creation and lookup; the fields allowed on the public card; retired labels.
backup.js, scheduler.js	Database dump and file copy, verification, restore support; hourly job runner.
provision.js	Creating the standard set of 194 accounts with temporary passwords.
assistant.js, assistantGuide.js	The AI assistant: connection to the configured OpenAI-compatible service, the four read-only lookups (search_assets, count_assets, get_asset, my_work) run with the user's scope, the server-side instructions and the built-in guide; one automatic retry when the service is overloaded (503); answers reduced to plain text.
inspections.js, siteCheck.js, sectorFields.js, photos.js, csv.js, constants.js, jimma.js	Inspection rules, on-site checks, sector-specific fields, image handling, safe CSV writing, enumerations, Jimma City boundary.

4 Request handling

4.1 Middleware order

The order in `backend/src/app.js` matters for security; it is listed here exactly as built.

1. `trust proxy = loopback`: the client address is taken from X-Forwarded-For only when the request comes from a proxy on the same machine.
2. `x-powered-by` disabled.
3. helmet with a custom Content Security Policy (section 4.2) and HSTS when `PUBLIC_URL` starts with https.
4. Permissions-Policy header: camera and geolocation for this site only; microphone, payment, USB, serial and Bluetooth off.
5. CORS: only `FRONTEND_ORIGIN` may call the API from a browser.
6. JSON body parser limited to 1 MB.
7. For `/api`: Cache-Control no-store, then a rate limit of 600 requests per minute per address.
8. For `/api/auth/login`: 20 per minute per address. For `/api/public`: 120 per minute per address.
9. Public endpoints: health, enumerations, translations.
10. Route modules. Each module applies `authenticate` (except the public card) and then its permission and area checks.
11. 404 for any other `/api` path; static files and the single-page application for everything else.
12. Error handler (section 7).

4.2 Security headers as sent

Header	Value
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self' https: 'unsafe-inline'; img-src 'self' data: blob: https://*.tile.openstreetmap.org https://server.arcgisonline.com; connect-src 'self'; font-src 'self' https: data:; frame-src https://www.youtube-nocookie.com; worker-src 'self' blob:; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'self'; script-src-attr 'none'; upgrade-insecure-requests (HTTPS only)
Strict-Transport-Security	max-age=31536000; includeSubDomains (only when PUBLIC_URL is https)
X-Content-Type-Options	nosniff

Header	Value
X-Frame-Options	SAMEORIGIN
Referrer-Policy	strict-origin-when-cross-origin
Cross-Origin-Opener-Policy / -Resource-Policy	same-origin / same-origin
Permissions-Policy	camera=(self), geolocation=(self), microphone=(), payment=(), usb=(), serial=(), bluetooth=(), interest-cohort=()
Cache-Control (API)	no-store; Pragma: no-cache

4.3 Authentication

A session token is a JWT signed with HS256 using `JWT_SECRET` (at least 16 characters, refused at start-up otherwise; 48 random bytes recommended). It carries only the user id (`sub`), the token version (`tv`) and the expiry (8 hours by default). On every request the server:

1. verifies the signature and expiry;
2. loads the user; refuses if the account is inactive;
3. compares the token version with `users.token_version`; sign-out, password change, role change, deactivation and "end sessions" all increase it;
4. refuses if `users.last_seen_at` is more than `SESSION_IDLE_MINUTES` (30) ago, otherwise updates it at most once a minute;
5. loads the user's area and the role's permissions;
6. refuses everything outside `/api/auth/` while the user must still replace a temporary password.

Sign-in counts failures in `users.failed_logins`; at 5 the account gets `locked_until` = now + 15 minutes. Unknown usernames are counted in memory with the same limits, and a dummy bcrypt comparison keeps the response time the same, so the two cases cannot be told apart.

5 API reference

All endpoints are under `/api` and exchange JSON unless stated otherwise. The route modules contain 130 handlers and `app.js` adds 4 public ones; all are listed below, grouped by area. "In area" applies to every endpoint that takes an asset or user identifier.

Public (no sign-in)

Method	Path	Who may call it	Purpose
`GET`	`/api/health`	Anyone	Liveness check: {ok: true}.
`GET`	`/api/enums`	Anyone	Allowed values for statuses, conditions, roles and other lists.
`GET`	`/api/i18n, /api/i18n/:lang`	Anyone	Available languages; one language's dictionary.
`GET`	`/api/public/assets/:token`	Anyone (120/min)	Public asset card. 404 unknown, 410 label replaced.
`GET`	`/api/public/assets/:token/photos/:id`	Anyone (120/min)	A photo of that asset only.

Authentication (/api/auth)

Method	Path	Who may call it	Purpose
POST	/login	Anyone (20/min)	Sign in; returns token and user.
GET	/me	Signed in	Current user, permissions and area.
POST	/logout	Signed in	Ends the session on the server.
POST	/change-password	Signed in	Requires the current password; ends other sessions.
POST	/language	Signed in	Saves the interface language.
PUT / DELETE	/photo	Signed in	Sets or removes one's own profile photo.
GET	/photo/:id	Signed in, may see that user	A user's profile photo.

Assets (/api/assets)

Method	Path	Who may call it	Purpose
GET	/	Signed in	Scoped, filtered, paged list.
GET	/export.csv	asset.export	The same list as CSV (formula-safe).
POST	/	asset.create	Register an asset (Sector Administrator, own sector and Kebele).
GET	/:id	In area	Full record; money fields only with financial.view.
PATCH	/:id	asset.edit, in area	Change details; not when disposed.
DELETE	/:id	Creator, draft only	Delete a never-submitted draft.
POST	/:id/workflow	Per step	Submit, verify (asset.review), approve (asset.approve), return.
POST	/:id/status	asset.edit, in area	Change status with a reason.
POST	/:id/utilizations, /:id/utilizations/:uid/end	asset.edit	Record or end a use of the asset.
POST / PATCH	/:id/maintenance, /:id/maintenance/:mid	maintenance.record	Record and update maintenance.
POST	/:id/inspections	inspection.record	Record an inspection (Inspectors).
POST	/:id/follow-ups, /:id/follow-ups/:fid/close, /reopen	Per rule	Follow-up actions after failed inspections.
POST	/:id/transfers	request.create	Raise a transfer request.
POST	/:id/valuations	financial.edit	Record a valuation.
POST	/:id/documents	asset.edit	Upload a document or photo (multipart, 10 MB).
GET	/:id/documents/:did/download	In area; money documents need financial.view	Download as attachment; logged.
PATCH / DELETE	/:id/documents/:did	asset.edit	Change or remove a document; logged.
GET	/:id/photo	In area	The asset's main photo.

Asset operations (/api/assets)

Method	Path	Who may call it	Purpose
GET	/map	Signed in	Assets with coordinates for the map, scoped.
GET	/:id/qr.svg, /:id/qr.png	In area	QR code image for the label.
POST	/:id/label/replace	asset.edit	Replace a lost or damaged label; the old token stops working.
POST	/scan	Signed in	Resolve a scanned code to an asset in area; logs the scan.
GET	/:id/scans	In area	Scan history.
PUT	/:id/acquisition	asset.edit	Acquisition and procurement details.
POST	/:id/assignments, /:id/ownership-change, /:id/disposal	request.create	Raise the corresponding request.
GET	/assignments/current	Signed in	Who holds what now, scoped.
POST	/:id/revenues	financial.edit	Record income.
POST	/:id/archive, /:id/unarchive	asset.edit	Archive a disposed asset or bring it back to the list.
GET	/follow-ups/open	Signed in	Open follow-up actions, scoped.
GET	/duplicates/scan	asset.export	Probable duplicate records.
GET / POST	/import/template.csv, /import	import.run	CSV template; bulk import (all rows checked first).

Requests and reservations

Method	Path	Who may call it	Purpose
GET	/api/requests	Signed in	Requests in area, filtered by status and type.
POST	/api/requests/:id/review	request.review	First step (Kebele).
POST	/api/requests/:id/approve, /reject	request.decide	Final decision (Kifle Ketema).
POST	/api/requests/:id/cancel	Requester	Withdraw a pending request.
GET	/api/reservations, /availability	Signed in	Reservations; free periods for an asset.
POST	/api/reservations	reservation.request	Request a reservation; overlap refused.
POST	/api/reservations/:id/approve, /reject, /cancel	reservation.decide / requester	Decide or withdraw.

Physical verification (/api/verification)

Method	Path	Who may call it	Purpose
GET / POST	/	Signed in / verification.manage	List campaigns; open a campaign for one's area.
GET	/:id	In area	Campaign with items and results.

Method	Path	Who may call it	Purpose
POST	/:id/record	verification.record	Record Found / Missing / Wrong location / Duplicate.
POST	/items/:itemId/resolve	verification.manage	Correct or dismiss a discrepancy (not by the person who found it).
POST	/:id/close	verification.manage	Close the campaign.
GET	/:id/report.csv	In area	Expected versus found, as CSV.

Finance, analytics, dashboard

Method	Path	Who may call it	Purpose
GET	/api/finance/overview, /valuations, /revenues, /backlog, /assets-search	financial.view	Money views, scoped.
POST	/api/finance/valuations, /revenues	financial.edit	Record from the finance page.
GET	/api/analytics	analytics.view	Rule-based indicators, scoped.
GET	/api/dashboard, /queue, /inspectors, /performance	Signed in (per role)	Dashboard figures, work queue, Inspector follow-up, performance by area.

Monthly reports (/api/monthly-reports)

Method	Path	Who may call it	Purpose
GET	/, /reminder, /team	Signed in	Own reports; current reminder; team status for a superior.
GET	/preview/:periodKey	Signed in	Figures the system would freeze.
POST	/:periodKey/submit	Signed in, has a superior	Send the report (freezes the figures).
GET	/report/:id	Sender or receiver	One report.
POST	/report/:id/accept, /return	Receiver	Decide; return needs a reason.

Notifications (/api/notifications)

Method	Path	Who may call it	Purpose
GET	/, /count	Signed in	Own notifications; unread count.
POST	/:id/read, /read-all	Signed in	Mark own notifications read.
GET / PUT	/rules, /rules/:type	system.manage	Notification rules.
POST	/run	system.manage	Run the alert job now.

Users (/api/users)

Method	Path	Who may call it	Purpose
GET	/	Account managers; Internal Auditor	Only accounts the caller manages (all, read-only, for the Auditor).

Method	Path	Who may call it	Purpose
POST	/	user.manage*	Create an account of the role the caller manages; returns the temporary password once.
GET / PATCH	/:id	Manager of that account	Read or change (role, area, active, reset password).
GET	/:id/history	Manager; Internal Auditor	Account history.
POST	/:id/end-sessions	Manager of that account	End all sessions of that user.
PUT / DELETE	/:id/photo	Manager of that account	Set or remove that user's photo.

AI assistant (/api/assistant)

Method	Path	Who may call it	Purpose
GET	/status	Signed in	Whether the assistant is switched on.
POST	/chat	Signed in (10/min per person)	{messages: [{role: user|assistant, content}]}, at most 12 turns of 2000 characters; returns {answer}. Read-only lookups in the caller's area.

Master data, administration, audit, integration

Method	Path	Who may call it	Purpose
GET	/api/master/lookups, /api/master/:resource	Signed in	Lists for forms (Kifle Ketemas, Kebeles, sectors, ...).
POST / PATCH	/api/master/:resource, /:resource/:id	master.manage	Add or change master data.
GET	/api/admin/health	system.manage	Database, disk, last backup, configuration warnings.
GET / PUT	/api/admin/workflows, /permissions/:role, /settings	system.manage	Workflows, role permissions, settings.
GET / POST	/api/admin/backups, /backups/:id/verify	system.manage	Backup history, new backup, verify.
GET / POST	/api/admin/api-keys, /api-keys/:id/disable	system.manage	Create (key shown once) and disable API keys.
GET	/api/audit	audit.view	Audit log, searchable.
GET	/api/integration/v1/openapi.json, /assets, /assets/:code, /locations, /sectors, /categories	X-API-Key	Read-only data for City systems; no money.

5.1 Response format

A successful call returns the resource or a list object. A list returns {rows, total, page, page_size}. Errors always have the shape below; details is present only for field errors.

```
{
  "error": "Please correct the highlighted fields.",
  "details": { "phone": "Enter a valid phone number (e.g. +251 911 223 344 or 0911 223 344)." }
}
```

6 Database

6.1 General rules

- Engine InnoDB, character set utf8mb4, collation utf8mb4_unicode_ci (Amharic and Afaan Oromo text).
- Every table has an auto-increment primary key `id` except pure link tables and settings.
- Foreign keys are declared for every reference. Unique keys: `assets.asset_code`, `assets.public_token`, `users.username`, `api_keys.key_hash`, master data names within their parent.
- CHECK constraints (migration 012) keep money non-negative, coordinates inside valid ranges, dates in order and quantities positive.
- Times are stored in UTC and shown in the user's time zone (East Africa Time).

6.2 Tables

Table	Purpose	Key columns
users	Accounts	username (UK), first/father/grandfather name, role, is_active, password_hash, must_change_password, temp_password_expires_at, token_version, failed_logins, locked_until, last_seen_at, phone, email, language, photo
user_kifle_ketemas, user_kebeles, user_sectors	A user's area	user_id, kifle_ketema_id / kebele_id / sector_id
role_permissions	Changes to the default permissions	role, permission, allowed
kifle_ketemas, kebeles, sectors, departments, asset_categories, asset_types	Master data	name, parent reference, working-label flag
assets	The register	asset_code (UK), name, category/type, sector/department, kifle_ketema/kebele, location_detail, latitude, longitude, ownership, responsible_officer, current_holder, condition, status, approval_state, acquisition_date, acquisition_cost, current_valuation, useful_life_years, salvage_value, public_token (UK), details (JSON, sector fields), created_by, submitted_by, reviewed_by, approved_by, disposed_at, archived_at
utilizations	How an asset is used	use_type, user_party, start/end, generates_revenue
maintenance_records	Maintenance	date, kind, cost, provider, status, next date
inspection_records	Inspections	date, result, condition_found, next date, recorded_by
follow_up_actions	Actions after failed inspections	description, due_date, status, resolution
asset_transfers, asset_assignments, asset_ownership_history	Movement and responsibility history	from/to, dates, reason, authorised_by, basis
asset_acquisitions, asset_disposals	Start and end of life	method, references, supplier, value, recipient
asset_valuations, asset_revenues	Money	value / amount, dates, method, payer, reference
asset_reservations	Bookings	requester, start_at, end_at, status

Table	Purpose	Key columns
asset_documents	Files	doc_type, title, original_name, stored_name (random), mime_type, size_bytes, related record
asset_site_checks	On-site confirmation during registration	result, latitude, longitude, photo, checked_by
approval_requests, workflow_configs	Requests and their rules	request_type, payload (JSON), status, requested/reviewed/decided by and at; steps
verification_campaigns, verification_items	Stock-taking	expected vs found location and condition, result, discrepancy_status, resolution
scan_events, asset_retired_labels	Label scans; replaced labels	context, user, coordinates; old token
notifications, notification_rules	Messages and their rules	type, title, link, dedupe_key, is_read, email/sms status; enabled, lead_days, email, sms
monthly_reports, report_schedules	Reporting	period_key, user_id, role, status, snapshot (JSON), version, decided_by
audit_logs	Audit trail (insert-only)	user_id, username, action, entity, entity_id, asset_id, summary, changes (JSON), created_at
api_keys	Integration keys	name, key_prefix, key_hash (SHA-256), is_active, last_used_at
backups, system_settings	Operations	filename, size, status, verified_at; key, value

6.3 Migrations

No.	Migration	No.	Migration
001	Base schema	011	Public token for QR cards
002	Smart-city tables (acquisition, verification, notifications ...)	012	CHECK constraints
003	User language	013	Retired labels
004	Forced password change	014	Approval state timestamps
005	On-site checks	015	Requests through the Kebele
006	Monthly reports	016	Follow-up checks
007	Reporting through the chain	017	Area centres for the map
008	Profile photos	018	Name parts (first, father, grandfather)
009	Sector details	019	Session security (lockout, idle, token version)
010	Asset type "other"	020	Temporary password expiry

7 Validation and error handling

7.1 Input rules

Kind of field	Rule (server)	Behaviour in the form
Person's name	Letters of any script (Latin, Ethiopic), spaces, full stop, apostrophe, hyphen; no digits; 1 to 80 characters per name part	Other characters cannot be typed
Username	Made by the system: the first name in lower case; if taken, first name and father's name; then a number	Not typed by the user
Phone	Optional +, 9 to 15 digits, no letters	Digits only; stops at 10 digits for 09/07 and 12 for +251
E-mail	Standard address format, at most 254 characters	Checked when leaving the field
Password	8 to 128 characters with upper, lower, digit and symbol	Strength shown while typing
Money (ETB)	Non-negative, at most two decimals, upper limit per field	Digits and one decimal point only
Dates	Valid calendar date; not in the future where it records something that happened; end after start	Date picker
Coordinates	Inside the Jimma City boundary	Filled from the phone's GPS or the map
Enumerations	One of the allowed values	Drop-down lists
Free text	Trimmed; maximum length per field (typically 200 or 2000)	Counter near the limit
Files	PDF, JPG, PNG, WebP, DOC(X), XLS(X); content must match the type; 10 MB; one file per request	Picker limited to these types

7.2 HTTP status codes

Code	When
400	Invalid input (with per-field details), malformed JSON, file too large, a referenced value does not exist.
401	Not signed in, token invalid or expired, session ended (sign-out, password change, idle), account inactive.
403	Signed in but the role lacks the permission; temporary password must be replaced first; temporary password expired; account deactivated (at sign-in).
404	Record does not exist or is outside the caller's area; unknown API path.
409	Conflict: duplicate unique value, record in use, reservation overlap, invalid state change.
410	A replaced QR label was scanned.
429	Rate limit exceeded or account temporarily locked; Retry-After header is sent.
500	Unexpected error. The body is only "Internal server error."; the detail goes to the server log.

8 Configuration

All settings are environment variables, read from `backend/.env` at start-up. The file must be readable only by the service account and must never be committed to version control.

Variable	Default	Meaning
PORT	4000	Port the Node.js process listens on (bind to 127.0.0.1 behind the proxy).
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	127.0.0.1, 3306, -, -, jimma_ams	Database connection.
DB_NAME_TEST	jimma_ams_test	The only database the tests are allowed to empty.
JWT_SECRET	(required)	Token signing secret; at least 16 characters, 48 random bytes recommended.
JWT_EXPIRES_IN	8h	Maximum session length.
SESSION_IDLE_MINUTES	30	Inactivity after which a session ends.
TEMP_PASSWORD_DAYS	7	Validity of a temporary password.
API_RATE_PER_MINUTE, LOGIN_RATE_PER_MINUTE, PUBLIC_RATE_PER_MINUTE	600, 20, 120	Rate limits per address.
SEPARATION_OF_DUTIES	true	Must stay true in production; false only for a one-person pilot.
PUBLIC_URL	(none)	Public https address; turns on HSTS and puts card links in QR codes.
FRONTEND_ORIGIN	http://localhost:5173	Allowed browser origin for CORS (development).
UPLOAD_DIR	uploads	Folder for files; outside the web root.
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASSWORD, SMTP_FROM	(none)	E-mail; disabled when SMTP_HOST is empty.
SMS_GATEWAY_URL, SMS_GATEWAY_TOKEN	(none)	SMS; disabled when empty.
BACKUP_DIR, MYSQLDUMP_PATH, AUTO_BACKUP	backups, mysqldump, false	Backups.
AI_BASE_URL, AI_API_KEY, AI_MODEL	(none)	AI assistant service (OpenAI-compatible); the assistant is off when not set. Examples in .env.example: Gemini, Groq, Ollama.
AI_REASONING_EFFORT	(none)	none, low, medium or high: how long a "thinking" model (Gemini) reasons before answering. low is used with Gemini (answers in about 3 to 8 seconds).
AI_RATE_PER_MINUTE	10	Questions a minute per person.

9 Background jobs

The scheduler starts 30 seconds after the server and then runs every hour. Each run is safe to repeat: notifications carry a de-duplication key so the same alert is not sent twice.

Job	What it does
Alerts	Finds maintenance and inspections due within their lead time or overdue, expiring documents and leases, critical assets, and creates notifications by the rules.
Monthly report reminders	Three days before each deadline, and after it, notifies the sender and their superior.
Inspector coverage	Warns when a Kifle Ketema has no active Inspector.
Daily backup (optional)	With AUTO_BACKUP=true, takes one database and file backup per day and verifies it.

10 Front-end design

- **Routing.** React Router; each role's menu is defined in `App.jsx`. A page a role cannot use is not in its menu, and the server refuses it anyway.
- **Server access.** All calls go through `api.js`, which adds the token, turns error answers into readable messages, and signs the user out on 401.
- **Session.** The token is kept in sessionStorage. The client also signs the user out after the idle time, so an unattended screen does not stay open.
- **Forms.** A shared form system (`components/ui.jsx`) applies the input rules while typing and shows server field errors next to the field.
- **Offline.** `offline.js` keeps inspection and verification results in a device outbox when the network is down and sends them when it returns; the service worker caches only the application shell.
- **Languages.** Texts are looked up through `t()` in dictionaries served by the backend; untranslated texts fall back to English.
- **No inline scripts.** The build produces only external script files, so the CSP can forbid inline scripts entirely.

11 Build, installation and operation

11.1 Installation

```
# 1. prerequisites: Node.js 20+, MySQL 8, nginx
# 2. database and user (in MySQL)
CREATE DATABASE jimma_ams CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'jimma_ams'@'localhost' IDENTIFIED BY '<long random password>';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES ON jimma_ams.* TO
'jimma_ams'@'localhost';
# 3. application
npm run setup                     # install backend and frontend packages
cp backend/.env.example backend/.env && chmod 600 backend/.env   # then edit
npm run migrate                   # create / upgrade tables
npm run seed                      # Kifle Ketemas, Kebeles, sectors, categories, rules
npm --prefix backend run create-admin -- admin "First Name"      # first Super Administrator
npm run build                     # build the web client
npm start                         # or run as a systemd service
```

After migration, CREATE, ALTER, INDEX and REFERENCES can be revoked from the application user and granted again only for upgrades.

11.2 Operations commands

Command (in backend)	Use
npm run migrate	Apply new migrations after an upgrade (safe to repeat).
npm run provision-admins [-- --dry-run]	Create the standard 194 accounts; temporary passwords go to a CSV file that must be handed out and deleted.
npm run reset-password --	Print a new temporary password once (for a locked-out administrator).
npm run backup	Database and files backup now.
npm run restore -- --yes	Restore the database from a backup.
npm test	Run the 265 tests against DB_NAME_TEST.

11.3 Testing strategy

The tests start the real Express application in-process and call it over HTTP with the same requests a browser would send, against a real MySQL test database that is migrated and filled with fixtures before each file. This tests the permission checks, the area scope, the validation and the database constraints together, which is where security mistakes usually hide. Tests run one file at a time to keep them independent. The full suite takes about 25 minutes on a development laptop; it passed completely (265 of 265) on 25 October 2026.

11.4 Coding conventions

- Every route that changes data parses its input with a zod schema first and writes an audit entry.
- Rules used by more than one route live in `lib/`, never copied.
- No string concatenation into SQL; the few raw fragments contain only column or table names fixed in the code.
- Comments explain why, not what; messages to users are complete sentences and translatable.